Automated Media Tiering System for Azure Blob Storage

Oliwier Kaminski 52321446

December 5, 2025

1 Introduction

During my work-based placement, I worked in Elementz: an organisation operating within the subsea asset-integrity and inspection management domain. In this company, large volumes of media are accessed daily for assessment, reviews, retention, and audits.

This organisation relies heavily on the Microsoft Azure Blob Storage cloud service as an archive and storage for the media Microsoft Corporation (2025*b*,*a*). In this case, media volumes consist of several terabytes of blobs, distributed across numerous storage containers. Due to Elementz being in a data-heavy industry, the storage requirements are accelerating as they bring more clients, resulting in significant cost pressure and operational challenges.

My role during this placement was equivalent to that of a software engineer working in cloud service optimisation. My responsibility was to explore solutions to minimise storage costs while mitigating the risk of slow accessibility and high operational costs.

2 Problem Definition

2.1 The Challenge: Intelligent Storage Tier Optimisation

The challenge I was tasked with included finding a way to extract Azure logs and blob metadata, and using them in an algorithm that would automatically manage and optimise Azure blob tiering. Azure offers several access tiers - Hot, Cool, Cold and Archive - each with different costs, performance, and retrieval characteristics Microsoft Corporation (2025*a*). Despite Azure's efficiency the following problems may occur:

- Azure's native lifecycle rules lack contextual awareness and often lead to misclassification. Some files might have only been accessed at the beginning of their lifetime, and yet they would remain in Hot - the most expensive tier.
- Some critical files that may be required for compliance could be placed in colder tiers if they have low access rates. This results in delays and additional costs upon reading them.

This creates a conflict between minimising cost and preserving access performance for critical media.

Cloud providers such as Microsoft explicitly encourage organisations to optimise their storage tier decisions and to combine access tiers with lifecycle policies Microsoft Corporation (2025*c*,*d*). Academic work on tiered cloud storage also highlights that intelligent data placement between hot and colder tiers can yield significant cost reductions when based on access patterns rather than

static rules Erradi & Mansouri (2020), Khan, Matskin, Prodan, Bussler, Roman & Soylu (2024). In practice, this optimisation requires a deep understanding of pricing models, access patterns, and file relevance. The absence of these qualities may lead to unnecessary storage expenses, retrieval delays for important media, manual labour overhead, and a lack of transparency around storage behaviour.

Thus, there was a clear need for a rule-based and context-aware tier optimisation system that could be integrated with both organisational workflows and Azure blob cloud storage.

2.2 Significance and Impact on the Business

Solving this problem provides several business benefits. Reducing tier heat (e.g. Hot to Cool) can significantly lower monthly storage expenses. Archive, being the coldest tier, offers further reductions but with costly operational risks. Microsoft's own documentation and best-practice guidance emphasise that appropriate use of blob access tiers and lifecycle management is one of the primary levers for optimising cloud storage costs Microsoft Corporation (2025*a*,*d*,*c*). Research on automated and rule-based tiering similarly reports measurable reductions in storage spend when access-aware policies are used instead of static tier assignments Khan, Matskin, Prodan, Bussler, Roman & Soylu (2024), Erradi & Mansouri (2020).

Correctly tiered media also reduces latency for critical workflows - for example, engineers reviewing new defects need instant Hot tier access. Tiering is part of a broader strategy of Information Lifecycle Management (ILM), which treats data and its associated metadata as assets that should be stored and protected in a way that reflects their current business value and risk profile Sheldon (2022).

This project also forms the first step towards an automated, metadata-driven policy framework and provides a platform for predictive storage optimisation in the near future.

2.3 Constraints Shaping the Problem Space

The solution was shaped by several requirements and constraints. The most significant limitation I came across was that, in practice, I had no reliable access to per-blob access logs due to the storage accounts using Hierarchical Namespace (ADLS Gen2) - which doesn't provide logs. Potential workarounds using services such as Event Grid, CDN, or Front Door to capture a subset of blob events Microsoft Corporation (2024) could have provided partial telemetry, but these options were unavailable within the constraints of the Azure for Students subscription used for development.

Another constraint arose from metadata availability. Several components of the tier optimisation algorithm depend on blob metadata (as per the project requirements) - such as criticality, relevance, and historical links. However, these values also could not be retrieved dynamically from Azure.

Even if logging were available, Azure only records telemetry after diagnostics are enabled. This means you cannot retroactively analyse past logs, as they do not exist.

2.4 Background Research and Prior Attempts

Before designing the solution, I conducted a review of the most relevant technical approaches to cloud-storage optimisation. The most applicable sources were covered in the Azure Lifecycle Management (LCM) policies, which provide rule-based tier transitions between access tiers and deletion actions Microsoft Corporation (2025*c*,*e*), N2WS (2025). While Azure LCM is useful for broad archival patterns, it operates primarily on simple usage heuristics. It does not incorporate access intelligence, rich metadata, or business-context signals - factors that would later become essential in this project's design.

In parallel, I analysed Azure's pricing model for its set of tiers [Hot, Cool, Cold, Archive], including both capacity and operational read costs. Azure's documentation and recent cost-modelling research emphasise that storage, transaction, and network charges interact in non-trivial ways, and that read operations can dominate the total cost for some tiers and workloads Microsoft Corporation (2025*a*), Khan et al. (2024). This means that an apparently Cold blob may still be cheaper to retain in Hot storage if expected access volumes are non-trivial. This pricing complexity underscored the need for a decision model capable of balancing cost, usage, and risk.

I also surveyed research on cloud-storage optimisation, including work on multi-tier storage and hierarchical data placement. Recent studies on automated tiering and storage-object classification provided concrete examples of how access frequency, object size, and metadata can be combined to drive tier decisions Erradi & Mansouri (2020), Khan, Matskin, Prodan, Bussler, Roman & Soylu (2024). Academic literature on decision-support systems and metadata-driven storage classification reinforced the value of combining access patterns with contextual metadata, supported by ILM guidance, which consistently emphasises that effective strategies incorporate relevance, criticality, and workflow implications in addition to file size or age Sheldon (2022), Khan et al. (2024).

Prior work consisted of manually moving and managing blobs at the request of the client. Assets that may have been decommissioned or sold would be moved to Archive, while the ones that would remain in use would stay in their tiers. Blobs still retained some metadata, meaning it was possible to change their tiers based on that information. This method however is impractical with storage containers that house millions of blobs due to the labour overhead that would come with it.

Collectively, this research made clear that existing tools and prior approaches were insufficient for the problem space. The company policies lacked the required depth and their solutions did not account for contextual factors. This justified the development of a custom algorithmic solution.

3 Approach and Methodology

3.1 Overall Methodological Strategy

The project addresses the practical research question: *"Given limited telemetry and complex pricing, how can we construct an explainable, metadata-aware decision model for Azure blob tiering?"* To answer this, the strategy combined systems engineering, design science, and iterative prototyping. The work began with exploratory research to understand Azure's storage model, pricing strategies, and the specific constraints introduced by Azure's architecture and data limitations. This established the technical boundary within which the solution had to operate.

The problem was then decomposed into several factors - the UI, Azure access layer, pricing engine, mock usage generator, and optimisation algorithm. Treating these as separate subsystems made it possible to design and test each one independently while making sure they would integrate together into a single solution.

Following established design science principles, the project centred on building functional artefacts that addressed the identified problem: a pricing engine reflecting real Azure costs, a deterministic usage simulator to compensate for the lack of telemetry, and an algorithm capable of combining pricing, metadata and usage into a tier recommendation system. Design science research guidelines emphasise iterative build–evaluate cycles Van der Merwe et al. (2020). As a result, these components were refined through iterative prototyping. Synthetic scenarios were used to test how the algorithm responded to different patterns of access, metadata and blob characteristics. Insights from each iteration built the foundation for refinements to penalties, scoring, and weight adjustments in the model.

Evaluation focused on verifying correctness, consistency and business alignment. This involved checking the cost outputs against Azure's pricing model and ensuring tier recommendations felt intuitive.

3.2 Why This Methodology Was Appropriate

The choice of methodology is supported by several strands of research. Decision-making literature highlights the importance of multi-criteria decision analysis (MCDA) when exploring trade-offs between equally important objectives Government Analysis Function (2024). MCDA provides a structured way to weight criteria, normalise inputs, and reason about alternatives when no single metric is sufficient Government Analysis Function (2024). This aligns directly with the challenge and focus of this project, where cost, performance and relevance all influence the final tiering decision.

In addition, Microsoft's own cost-optimisation guidance for Blob Storage stresses that any tiering logic must consider both capacity charges and read costs, as either factor can dominate depending on the tier and workload Microsoft Corporation (2025*d*,*c*). Recent cloud cost-modelling work reinforces this point by treating storage and transaction charges as separate dimensions within the objective Khan et al. (2024).

3.3 Breaking Down the Problem

To make the problem manageable, I broke the system down into a set of components. These components changed throughout development to accommodate overlooked issues and limitations.

- The Azure Access Layer - Responsible for handling secure, authenticated communication with Blob storage via tokenisation.
- The Pricing Engine - Pulls costing information from Azure regarding storage and read costs in each tier. These prices vary per region and account type.
- The Algorithm - Converts all inputs into a final tier recommendation.
- Web Application - Hosted on a web application using the Django framework.
- Evaluation Strategy - Consists of designing a series of synthetic blobs with varying metadata and size in order to test algorithm accuracy.
- Mock Usage Engine - Originally unplanned, however responsible for generating metadata and usage patterns to be fed into the algorithm.

This decomposition kept the system modular and simpler to develop, ensuring each part was refined independently and easily tested.

4 Solution Architecture

4.1 System Architecture

The system (Figure 1) is a Django web application that sits between the user's browser, Azure blob storage, and the tier optimisation system. When a user interacts with the UI, Django views handle the request, query Azure blob storage for blob details, and then call the mock engine to fill them with metadata. This, in conjunction with the pricing engine, is used to compute the optimal tier. The results are passed back into views, which get displayed on the browser.

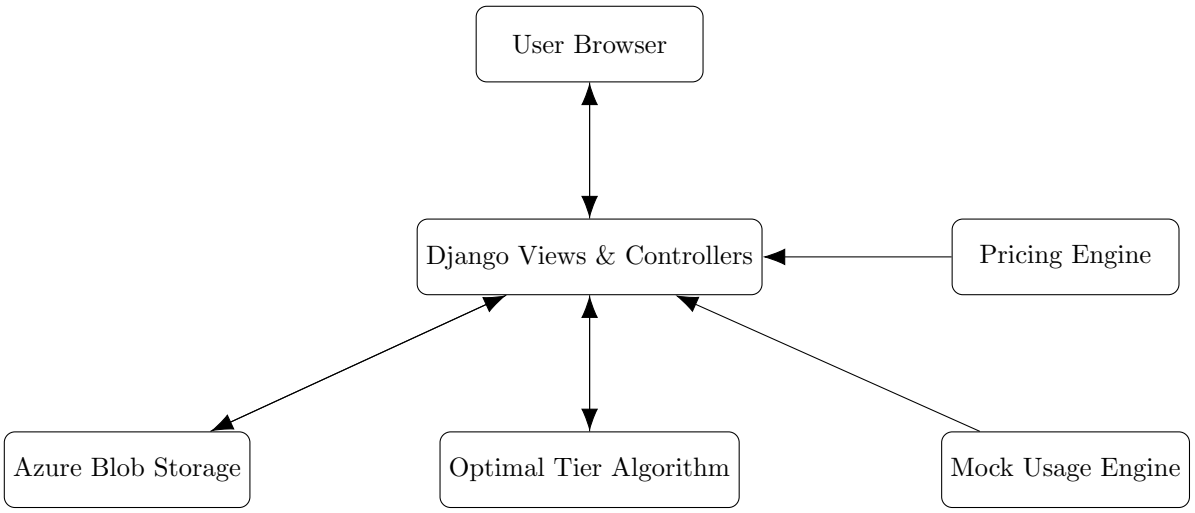

Figure 1: System Architecture Overview

These architectural choices were made deliberately. Django provides a stable foundation for future extension of workflows (e.g. integration with internal systems) while remaining easy to deploy and develop with . Re-using Azure's native identity and Blob APIs avoids additional infrastructure and reduces operational risk. Finally, separating the pricing engine, mock usage engine and optimisation algorithm into distinct services keeps the decision logic modular, testable, and replaceable - for example, a future LLM could be substituted for the current rule-based engine without rewriting the web layer.

4.2 Component Descriptions

The Django application forms the user-facing layer. It renders the homepage and blob details, accepts inputs such as which container to display, what prefix to search by and the page size. It displays a table of blobs with their current tier, simulated access counts, cost estimates, and optimal tier suggestions. It is responsible for handling single and bulk tier changes and integrates charts to visualise current, versus optimised monthly storage costs.

Azure integration is built around a module-level *BlobServiceClient*, authenticated via *DefaultAzureCredential*. The access layer allows the same code to be run both locally and in cloud environments by using CLI authentication. Through this client, the application is able to list blobs present in storage accounts and issue tier-change operations.

The pricing engine models Azure storage economics for the relevant region and account redundancy settings. It computes capacity and read costs, and a combined monthly cost for each tier.

Since no real per-blob access logs or metadata are available, a mock usage engine simulates usage histories and contextual signals. For each blob, it generates read counts over multiple time windows, age-related data, and metadata such as criticality, relevance, human-trigger likelihood, historical linkage, and planned activity.

The tier optimisation algorithm is the core decision-making component. It takes blob size, simulated access patterns, and metadata as inputs, applies the pricing engine's cost model, and outputs a recommended tier alongside a cost table. It is designed to be explainable and rule-based,

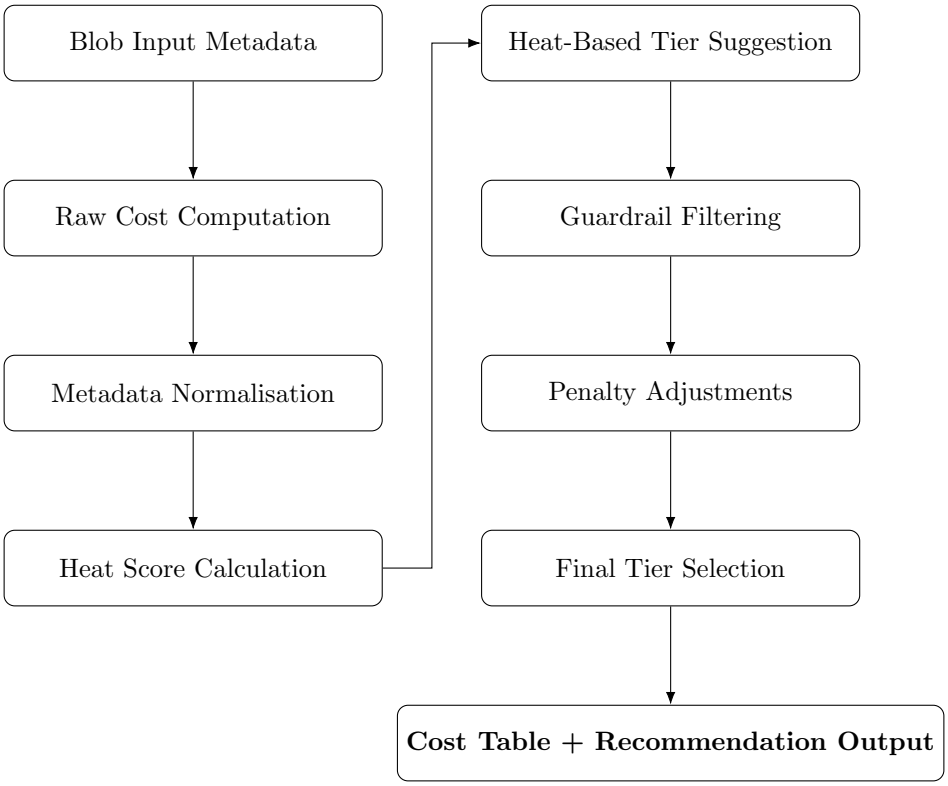

Figure 2: Detailed Algorithm Architecture

applying guardrails, penalties and heat scores so that recommendations are both economically valid and operationally reasonable.

4.3 Detailed Algorithm Architecture

Within the algorithm, processing follows a structured flow (Figure 2). The application first passes all inputs into the algorithm - blob size, read logs, and metadata. The raw monthly storage cost is computed afterwards for each tier by combining capacity and read-costs. Next, all metadata is normalised into a consistent 0-1 float scale. Based on these features, the algorithm derives a heat score that approximates how hot or cold the blob's expected future usage is.

Guardrails are applied to remove unsafe options (i.e. preventing highly critical blobs from being placed in colder tiers), and penalty functions adjust the effective cost of each remaining tier based on distance from the target tier, blob age, size, and historical characteristics. Finally, the tier with the lowest penalty-adjusted cost is selected, and the algorithm returns both the recommended tier and the cost breakdown.

4.4 Data Flow

The data flow (Figure 3) through the system starts when a user issues a request to the homepage with a specified container and optional prefix. Django receives the request, calls Azure Blob Storage

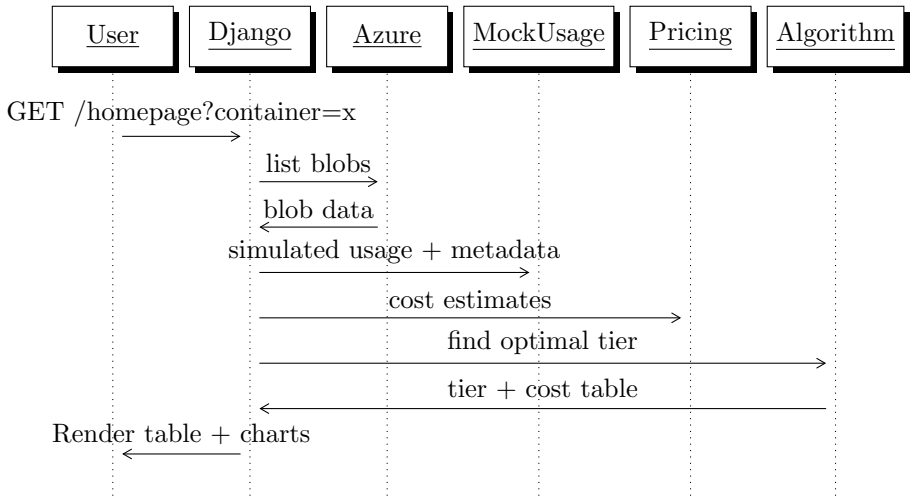

Figure 3: Data flow between user, Django, Azure and services

to list the matching blobs, and then for each blob invokes the mock usage engine to obtain simulated access windows and metadata, as well as the pricing engine to compute cost estimates. These inputs are passed into the optimisation algorithm, which returns the recommended tier and cost table for each blob. Django aggregates these results and renders them back to the user.

5 Implementation and Evaluation

5.1 Implementation Breakdown

The implementation closely followed the architectural structure outlined earlier. The central entry point is the *homepage* view, which coordinates both the blob-listing workflow and the tier-changing workflow. For GET requests, Django validates the container name, retrieves blobs from Azure, annotates each one with simulated metadata and cost information, and applies pagination before returning the results to the interface. POST requests handle both single and bulk tier changes.

Two utility modules contain the key computational logic. The pricing engine maintains Azure's capacity and read-operation prices for the set of tiers, and exposes functions for estimating storage cost, read cost, and combined monthly cost. It also normalises tier names returned by Azure so they map cleanly onto the internal tier model.

The mock usage engine provides deterministic simulated behaviour. It derives a stable random seed from a SHA-256 hash of the container and blob names, assigns each blob a behavioural profile, and then generates the daily read counts over a synthetic history window. A second helper produces the metadata, again using the same seeded random generator so the outputs remain reproducible. These values are attached to each blob through an annotation helper in the Django view.

The frontend is delivered through Django templates. The main *homepage.html* template renders the blob table, cost estimates, suggested tier changes, and two bar graph visualisations comparing current and optimised monthly spend. Several lightweight JavaScript helpers power the modals used for single-blob and bulk tier recommendations and generate brief 'reason for recommendation' explanations using metadata fields. The *blob_info.html* template provides a more detailed view of a

single blob, primarily for inspection and debugging.

5.2 The Tier Optimisation Algorithm

The tier optimisation algorithm combines cost modelling, metadata signals, and a structured decision process to produce an explainable recommendation for each blob. Its first stage computes the raw monthly cost of storing the blob in each tier, based on Azure's capacity and read-operation pricing. This provides the financial baseline for later adjustments.

Metadata supplied to the algorithm is then normalised to a consistent 0–1 scale, with age capped at two years to prevent disproportionately penalising very old blobs. These normalised attributes feed into a heat-scoring function that estimates the likelihood of future access. The score is calculated as a weighted combination of the metadata. Higher heat values imply a greater need for fast retrieval and therefore favour hotter tiers.

From the heat score, the algorithm identifies an initial target tier. This target is an anchor for the remaining decision logic. Before any tier can be selected, guardrails are applied to eliminate options that would be operationally inappropriate. For example, recently accessed blobs and those with high criticality or planned activity are prevented from being placed in the coldest tiers.

For the tiers that remain valid, the algorithm applies a set of penalty adjustments. A distance penalty discourages selecting tiers far from the heat-based target, while additional adjustments account for relevance, read-intensity, size, age, and historical linkage. These penalties allow the model to express nuances that raw cost alone cannot capture—for instance.

Once all penalties are added to the raw costs, the algorithm selects the tier with the lowest adjusted value. The output includes the recommended tier, a full per-tier cost table, and an estimate of the potential savings.

5.3 System Testing and Evaluation

The system was tested using a combination of unit tests, controlled synthetic scenarios, and performance checks. Unit tests were used to verify the reliability of core functionality, including container-name validation, blob uploads, tier-change operations, and the handling of Azure API exceptions.

To evaluate the algorithm's decision-making, I created a set of synthetic scenarios, each representing a different combination of size, metadata, and usage characteristics. During the first few iterations, the algorithm's output did not align with expectations. There were cases where highly critical blobs would be placed in Cold or Archive due to their low usage rates. As a result, hard guardrails were implemented and weights were adjusted to more closely reflect expected results with each iterations. The scenario sets can be found in the appendix.

Performance testing focused on the practical challenge of handling large containers. Pages containing more than a thousand blobs were used to validate that pagination, annotation, and cost computation remained responsive. Originally, pagination was designed in a way to only load necessary blobs that can be seen by the user; this would prevent massive containers from attempting to send all blobs, ultimately slowing down the application. The organisation however requested to be able to see total costs of containers and the total estimated savings after tier adjustments - which resulted in all blobs being loaded anyway. Even with massive containers, Azure API calls stayed within seamless latency bounds.

The system received biweekly reviews to assess the algorithm's completeness and behaviour. Each review generated targeted improvement suggestions and kept me on the right track during development. Overall, the final evaluation showed strong alignment between the system's outputs and the organisation's understanding of how media should be managed across storage tiers.

5.4 Strengths

A key strength of the system is its high level of explainability. Every recommendation is accompanied by a clear justification, allowing users to understand precisely why a particular tier was selected. In cases where the justification is not clear, the user is able to also see the metadata of the blob for clarity. The system is configurable, with weights, penalties, and guardrails designed to be adjusted all in one place as organisational priorities evolve, Azure pricing changes or as further optimisation is required.

Another advantage is that the solution remains fully Azure-native, using the platform's authentication model, storage APIs, and tiering mechanisms without introducing any other dependencies. This ensures compatibility with existing infrastructure and simplifies deployment.

The system also reflects organisational metadata, meaning its behaviour mirrors how media is actually used by each client. Safeguards prevent unsafe archival actions—ensuring that critical, or operationally important files cannot be placed into tiers that would compromise ease of access.

Finally, updating the blob to the optimal tier is intuitive and simple. The user is able to either make singular per-blob changes by searching for the name, or make container-wide adjustments.

5.5 Limitations

Despite its strengths, the system has several limitations. The mock usage engine, while necessary due to the absence of real access telemetry, cannot fully capture the nuances of true user behaviour or workload patterns. This means some recommendations may differ from those that would emerge if full historical logs were available.

Azure's pricing model is another source of uncertainty. Storage and transaction costs evolve over time, and adjustments in regional pricing or tier behaviours may require recalibration of the pricing engine and, in turn, the optimisation algorithm. I was also unable to find a way to export this data via a python package, meaning all costs are hard-coded in the program.

From a performance perspective, the front-end may exhibit latency when rendering extremely large blob listings, particularly when thousands of items are displayed on a single page. Although pagination mitigates this, highly data-dense environments could still experience delays.

Finally, some edge cases—especially those involving unusual metadata combinations—may require domain-specific tuning to keep the algorithm optimal. While the system is designed to be adaptable, achieving perfect alignment in these scenarios would require a very large set of guardrails, which is unfeasible due to the expanding client-base.

6 Reflection and Skills Development

6.1 Technical Skills Developed

Cloud architecture was a major focus, particularly in working with Azure Blob Storage and its access-tiering model. Since I have never worked with cloud architecture before, it was a steep learning curve. I gained practical experience with Azure's identity and authentication mechanisms, as well as the Azure SDK for Python. This strengthened my understanding of how cloud resources behave under different constraints and pricing models.

On the backend engineering side, building the Django application deepened my skills in structuring views, templates, and forms, and in implementing robust pagination and exception handling.

The most significant learning came from the algorithm design element of the project as this is the first time I was building my own algorithm. Developing an explainable tier-recommendation

model required breaking down a complex problem into smaller components, designing multi-criteria scoring functions, and embedding guardrails. Implementing a decision model that could balance cost, metadata, and operational factors was a valuable exercise..

Finally, the project strengthened my abilities in data modelling. Because real access logs and metadata were unavailable, I developed deterministic simulations for usage windows, relevance scores, and behavioural signals. This provided insight into how data can be modelled when direct telemetry is missing, and how synthetic data can still support development.

6.2 Transferable Skills Developed

The project also helped develop a range of transferable skills. Critical thinking played an important role throughout, particularly when interpreting Azure's pricing structures and identifying the practical implications hidden behind ambiguous documentation. This required balancing theoretical cost models with real operational constraints.

Clear communication became essential as the algorithm matured. I needed to translate complex decision logic into explanations that were understandable to non-technical users, both within the UI and during regular sprint meetings. This strengthened my ability to present technical reasoning in accessible terms.

Finally, collaboration with my industry partner helped shape the priorities and direction of the solution. Regular meetings ensured that technical decisions aligned with business needs and improved my ability to work iteratively, rely on feedback, and communicate progress effectively.

6.3 Obstacles and How I Overcame Them

Several obstacles shaped the development process. The most significant challenge was the complete absence of per-blob access logs since the optimisation algorithm depended on understanding access behaviour.

Another obstacle was the complexity of Azure's pricing model, which combines capacity charges, read-operation costs, and region-specific behaviours. Relying on static assumptions would have resulted in inaccurate or misleading tier recommendations.

Finally, the project surfaced performance challenges on the front end, particularly when rendering large containers with thousands of blobs. Without mitigation, this could lead to slow page loads and a poor user experience. To address this, I implemented pagination and prefix filtering, allowing users to narrow listings and ensuring the interface remained responsive where possible.

6.4 Personal and Professional Growth

This project broadened my understanding of how enterprise cloud systems operate in practice. I gained a clearer appreciation of how seemingly simple storage decisions can evolve into complex optimisation problems once factors such as pricing, access behaviour, and metadata. Designing a tier-recommendation algorithm without access to real telemetry also taught me how to justify and construct models carefully.

Beyond the technical insights, working through challenges - particularly Azure's logging limitations - required persistence, creativity, and patience to explore unusual solutions. Developing the mock usage engine and refining the algorithm through iterative testing strengthened my independence and problem-solving skills. Overall, the experience deepened both my technical confidence and my ability to navigate ambiguous problem spaces.

6.5 What I Would Do Differently

If I were to begin this project again, I would start by building a smaller-scale prototype of the algorithm much earlier. This would have created more time for iterative refinement, scenario testing, and deeper validation of the core decision logic. I would also communicate with the industry partner more frequently in the early stages to confirm that my priorities and interpretation of the tasks are aligned with the current sprint. Finally, I would take greater care with the project goal and translate it into a structured delivery plan, breaking the work down into clearer milestones, in order to better manage my time.

7 Conclusion

This project delivered a practical, explainable system for automating storage-tier decisions within Azure Blob Storage. It addressed a core organisational challenge: the lack of a mechanism to allocate media to appropriate tiers. By combining a pricing engine, a deterministic mock-usage model, and a rule-based optimisation algorithm within a Django web interface, the system provides clear recommendations.

Across six rounds of scenario testing and bi-weekly progress reviews, the app improved steadily. Early issues with the algorithm revealed inaccuracies such as excessive archival, insufficient sensitivity to metadata, and weak guardrail behaviour. Later iterations showed more accurate and stable tiering decisions, with misclassifications reduced but still present in edge cases. These refinements ensured the system's outputs aligned with operational expectations while remaining interpretable and safe.

Although limited by the absence of true access logs and by the variability of Azure pricing, the final system offers a reliable foundation for automated tier management. It demonstrates that even with incomplete data, meaningful optimisation can be achieved through structured heuristics and transparent decision models. The work provides a scalable starting point for future extensions—including integration with real telemetry or predictive analytics—and offers immediate value by supporting more consistent, cost-aware storage within the organisation.

Word Count: 4334

References

Erradi, A. & Mansouri, Y. (2020), 'Online cost optimization algorithms for tiered cloud storage services', *Journal of Systems and Software* **160**, 110457.

Government Analysis Function (2024), 'An introductory guide to multi-criteria decision analysis (mcda)', https://analysisfunction.civilservice.gov.uk/policy-store/an-introductory-guide-to-mcda/. Accessed 3 December 2025.

Khan, A. Q., Matskin, M., Prodan, R., Bussler, C., Roman, D. & Soylu, A. (2024), 'Cloud storage tier optimization through storage object classification', *Computing* **106**(11), 3389–3418.

Khan, A. Q. et al. (2024), 'Cost modelling and optimisation for cloud storage services', Preprint / technical report on cloud cost models. Cited for capacity vs transaction cost treatment.

Microsoft Corporation (2024), 'Azure blob storage as event grid source', https://learn.microsoft.com/en-us/azure/event-grid/event-schema-blob-storage. Accessed 3 December 2025.

Microsoft Corporation (2025*a*), 'Access tiers for blob data', https://learn.microsoft.com/en-us/azure/storage/blobs/access-tiers-overview. Accessed 3 December 2025.

Microsoft Corporation (2025*b*), 'Azure blob storage documentation', https://learn.microsoft.com/en-us/azure/storage/blobs/. Accessed 3 December 2025.

Microsoft Corporation (2025*c*), 'Azure blob storage lifecycle management overview', https://learn.microsoft.com/en-us/azure/storage/blobs/lifecycle-management-overview. Accessed 3 December 2025.

Microsoft Corporation (2025*d*), 'Best practices for using blob access tiers', https://learn.microsoft.com/en-us/azure/storage/blobs/access-tiers-best-practices. Accessed 3 December 2025.

Microsoft Corporation (2025*e*), 'Configure a lifecycle management policy for blob storage', https://learn.microsoft.com/en-us/azure/storage/blobs/lifecycle-management-policy-configure. Accessed 3 December 2025.

N2WS (2025), 'Azure storage lifecycle management: Quick tutorial and pro tips', https://n2ws.com/blog/azure-storage-lifecycle-management-pro-tips. Accessed 3 December 2025.

Sheldon, R. (2022), 'What is information lifecycle management (ilm)?', https://www.techtarget.com/searchstorage/definition/information-life-cycle-management. TechTarget, accessed 3 December 2025.

Van der Merwe, A., Gerber, A. & Smuts, H. (2020), Guidelines for conducting design science research in information systems, *in* B. Tait, J. Kroeze & S. Gruner, eds, 'ICT Education (SACLA 2019)', Vol. 1136 of *Communications in Computer and Information Science*, Springer, Cham, pp. 163–178.

Appendices

A Scenario Testing Tables

A.1 Scenario Set 1

ID	Blob scenario	Algorithm result	Expected optimal tier
1	Critical defect photo, accessed daily in last week, planned review next month.	Cool	Hot
2	Inspection video, last accessed 400 days ago, criticality low, no planned activity.	Hot	Archive
3	Engineering drawing, medium criticality, last access 3 days ago, human-trigger likely.	Cool	Hot
4	Historic survey video, no recent access (700+ days), relevance low.	Cold	Cold
5	Compliance PDF, high criticality, last access 90 days ago, legal retention required.	Cool	Hot
6	Legacy test image, no access in 2 years, no metadata, no historical links.	Hot	Archive
7	Training video, moderate relevance, accessed once in last 60 days.	Cool	Cool
8	Defect screenshot, high relevance, last access 10 days ago, planned activity true.	Cool	Hot
9	Drawing, low criticality, last access 370 days ago, some historical linkage.	Hot	Cold
10	Log export, low relevance, never accessed since upload.	Archive	Archive
11	Inspection report, high human-trigger index, accessed last week.	Cool	Hot
12	Calibration video, medium criticality, last access 150 days ago.	Archive	Cool
13	Archived inspection photo, no reads in 500 days, no planned activity.	Archive	Archive
14	General reference document, medium relevance, occasional access (every 3–4 months).	Hot	Cool
15	Defect timelapse, high criticality and historical linkage, last access 45 days ago.	Cool	Hot
16	Campaign dump, huge file, minimal access (1 read in 2 years), low criticality.	Archive	Cold
17	Email attachment capture, zero access, no metadata beyond age.	Archive	Archive
18	Inspection pack, medium relevance, last access 250 days ago, no future activity.	Archive	Cold
19	Photo used in external report, high relevance, last access 20 days ago.	Cool	Hot
20	Obsolete raw footage, age ¿ 730 days, low relevance, no links.	Archive	Archive

Table 1: Early checks of basic tiering behaviour and adjustment of basic weights.

A.2 Scenario Set 2

ID	Blob scenario	Algorithm result	Expected optimal tier
1	Defect cluster summary, medium criticality, high relevance, low recent access.	Hot	Cool
2	QA checklist template, moderate relevance, frequent human-trigger usage.	Hot	Hot
3	Internal note, low relevance, occasional use, age 30 days.	Cool	Cool
4	Inspection plan document, high planned activity, age 10 days.	Hot	Hot
5	Old survey, low criticality but strong historical linkage to active assets.	Cold	Cold
6	Image set, high human-trigger index, access every few weeks.	Cool	Hot
7	Defect evidence pack, criticality high, last access 120 days ago.	Hot	Hot
8	Generic guidance PDF, medium relevance, rare reads (twice a year).	Cool	Cool
9	Shared training bundle, high relevance but no recent use.	Cool	Cool
10	Legacy metrics export, age ¿ 2 years, no reads, no metadata.	Archive	Archive
11	Inspection replay snapshot, medium relevance, last read 20 days ago.	Cool	Hot
12	Structural drawing, medium criticality, occasional access, age 18 months.	Archive	Cool
13	Experiment screenshot, no historical linkage, minimal metadata, 1 read.	Hot	Cold
14	Audit summary, high criticality, last read 11 months ago.	Hot	Hot
15	Troubleshooting video, human-trigger likely during outages, no recent outages.	Hot	Cool
16	Configuration reference, moderate relevance, sporadic usage.	Hot	Cool
17	Deprecated documentation, low relevance, last accessed 400 days ago.	Cold	Cold
18	UI mock-up, age 600 days, no access, no planned use.	Archive	Archive
19	Multi-asset overview, medium relevance and historical linkage, slow but ongoing access.	Hot	Cool
20	Urgent incident report, high human-trigger and criticality, age 5 days.	Cool	Hot

Table 2: Adjusted sensitivity of the heat score to metadata and added guardrails against Archive.

A.3 Scenario Set 3

ID	Blob scenario	Algorithm result	Expected optimal tier
1	Campaign video, low criticality, no reads in 18 months.	Cold	Cold
2	Old ROV footage, moderate relevance for rare root-cause investigations.	Cold	Cold
3	Unedited raw inspection dump, no access since upload (2+ years).	Archive	Archive
4	Simulation output, medium relevance, last used 10 months ago.	Cold	Cold
5	Test dataset, no business value, kept only for possible debugging.	Cold	Cold
6	Time-lapse archive, occasional use for long-term patterns.	Hot	Cold
7	Asset baseline record, high historical linkage, low ongoing usage.	Hot	Cold
8	Internal training artefacts, rarely accessed, low criticality.	Hot	Cold
9	Vendor data dump, never accessed, low relevance.	Archive	Archive
10	Inspection stack, used intensively for first 3 months, then unused for a year.	Hot	Cold
11	Video, high criticality but only needed for rare legal queries.	Cool	Cool
12	Reference data, occasional access by analysts, medium criticality.	Hot	Cool
13	Compressed archive, no reads, age ¿ 2 years, no metadata.	Archive	Archive
14	High-frequency logs, compressed, occasionally revisited.	Hot	Cool
15	Processed inspection set, medium relevance, last access 250 days ago.	Cold	Cold
16	Historic engineering model, high historical linkage, no planned activity.	Hot	Cold
17	Test footage, never used, retained accidentally.	Archive	Archive
18	Internal QA recordings, last access 2 years ago, low criticality.	Hot	Archive
19	Campaign-level video assets, low planned activity, some historical linkage.	Hot	Cold
20	Archived defect collection, mildly relevant but rarely accessed.	Cold	Cold

Table 3: Large object-specific rules implemented for larger blobs and age interaction adjusted.

A.4 Scenario Set 4

ID	Blob scenario	Algorithm result	Expected optimal tier
1	Legal hold document, very high criticality, no reads in 2 years.	Hot	Hot
2	Safety incident report, high criticality, last access 14 months ago.	Hot	Hot
3	Regulatory audit trail, must remain instantly available during audits.	Hot	Hot
4	Contract PDF, legal retention required, low recent access.	Cool	Hot
5	Insurance evidence pack, high criticality, no planned activity but non-negotiable retention.	Hot	Hot
6	Internal policy document, medium criticality, used occasionally in reviews.	Cool	Cool
7	Supplier non-conformance record, sometimes revisited in disputes.	Cold	Cool
8	Management summary, high visibility but low direct reads.	Hot	Cool
9	Risk register export, high business importance, sporadic access.	Hot	Hot
10	Compliance checklist, often opened before audits, human-trigger high.	Hot	Hot
11	Legal opinion, rare access but high sensitivity.	Hot	Hot
12	Confidentiality agreement, medium relevance, occasional checks.	Cool	Cool
13	NDA, long-lived but rarely accessed.	Cool	Cool
14	Warranty document, needed only if defects arise.	Cold	Cool
15	Contract amendment, historically important, low recent use.	Cool	Cool
16	Safety guidance note, human-triggered when incidents occur.	Cool	Hot
17	Compliance matrix, heavily used during certain projects, idle outside them.	Hot	Hot
18	Risk escalation pack, used recently for a major issue.	Hot	Hot
19	Executive briefing, high visibility, likely to be re-opened in reviews.	Cool	Hot
20	Audit close-out report, medium criticality, accessed rarely post-closure.	Cold	Cool

Table 4: Ensuring critical, legal, and human-interaction artefacts stay sufficiently hot.

A.5 Scenario Set 5

ID	Blob scenario	Algorithm result	Expected optimal tier
1	Historic defect photo linked to active asset, no recent access.	Cold	Cold
2	Old inspection report referenced by multiple more recent inspections.	Cool	Cool
3	Baseline schematic for asset still in service, no recent reads.	Cold	Cool
4	Legacy inspection summary for decommissioned asset.	Archive	Archive
5	Engineering memo with strong linkage to current design decisions.	Cool	Cool
6	Survey pack, marked as "reference for future upgrades".	Cool	Cool
7	Commissioning record, linked to several open tickets.	Hot	Hot
8	Vendor manual, historically important but low usage, asset retired.	Archive	Archive
9	Baseline corrosion map for asset still monitored.	Cold	Cool
10	Configuration snapshot preceding a known incident, occasionally revisited.	Cool	Cool
11	Asset decommissioning report, no further use expected.	Archive	Archive
12	Pipework diagram with links from multiple inspection tasks.	Cool	Cool
13	Commissioning video with moderate historical linkage.	Archive	Cold
14	Old work instruction, replaced by newer version but kept for traceability.	Cold	Cold
15	Vendor test certificate linked to ongoing warranty.	Hot	Hot
16	Baseline photo set for asset still under surveillance.	Cool	Cool
17	Obsolete commissioning checklist, no links, no access.	Archive	Archive
18	Failure analysis report referenced by standards.	Cool	Cool
19	Old operating guideline, superseded but occasionally consulted.	Hot	Cool
20	Outdated CAD snapshot, no current projects relying on it.	Cool	Cold

Table 5: Handling historically linked blobs with greater care while biasing towards colder tiers.

A.6 Scenario Set 6

ID	Blob scenario	Algorithm result	Expected optimal tier
1	Moderately important drawing, occasional access, age 8 months.	Hot	Cool
2	Minor inspection note, low relevance, last read 5 months ago.	Cold	Cold
3	Defect close-out photo, medium relevance, last access 40 days ago.	Cool	Cool
4	Issue-tracking export, medium relevance, age 3 months.	Hot	Cool
5	Routine inspection report, low criticality, access every 6 months.	Hot	Cool
6	Medium-priority risk log, last access 20 days ago.	Hot	Hot
7	Minor vendor note, no reads since upload, low criticality.	Archive	Archive
8	Small video clip used occasionally for training, age 1 year.	Cool	Cool
9	Periodic status report, relevance waning, last access 10 months ago.	Cold	Cold
10	Misc attachment, unknown purpose, never accessed.	Archive	Archive
11	Engineering decision record, medium criticality, last access 3 months ago.	Cool	Cool
12	Low-value screenshot, no metadata, no access.	Archive	Archive
13	Recurring audit template, high human-trigger when audits run.	Hot	Hot
14	Operational note, access every few weeks, medium relevance.	Hot	Hot
15	Small CAD preview, last access 15 months ago, low criticality.	Cold	Cold
16	Old trend chart, occasionally checked, but age ¿ 2 years.	Archive	Cold
17	Problem description summary, linked to closed tickets only.	Cold	Cold
18	High-criticality escalation summary, last access 60 days ago.	Hot	Hot
19	Generic FAQ, accessed irregularly but still useful.	Hot	Cool
20	Older FAQ version, superseded, no direct access.	Archive	Archive

Table 6: Fine-tuning of weights and tier cut-offs to promote criticality and relevance.